Daily Quant Digest: 2026-10-15

Today's topics

Track	Topic	Difficulty
Probability & Statistics	Generating functions (PGF, MGF, characteristic functions); can you load two dice so the sum is uniform?	Medium
Derivatives	Repo, specialness, and the Treasury cash-futures basis trade	Medium
Algorithms & DS	Binary search on the answer: split array largest sum	Medium-Hard

1. Probability & Statistics: Generating Functions

1.1 Three transforms, one idea

A generating function packs an entire distribution into a single function, so that hard operations on distributions (convolution, mixing, limits) become easy operations on functions (multiplication, composition, pointwise limits).

Transform	Definition	Exists for	Main use
PGF	$G_X(s) = \mathbb{E}[s^X] = \sum_{n\ge 0} P(X=n)\, s^n$	non-negative integer X, at least for $\lvert s\rvert \le 1$	counting, dice, waiting times, branching
MGF	$M_X(t) = \mathbb{E}[e^{tX}]$	only if finite near $t = 0$	moments, tails (Chernoff), cumulants
CF	$\varphi_X(t) = \mathbb{E}[e^{itX}]$	**always** (since $\lvert e^{itX}\rvert = 1$)	limit theorems, inversion, heavy tails

They are the same object viewed differently: $M_X(t) = G_X(e^t)$ and $\varphi_X(t) = M_X(it)$ whenever both sides make sense. The **cumulant generating function** is $K_X(t) = \log M_X(t)$.

1.2 The four properties that do all the work

(a) Independence turns sums into products. If $X \perp Y$ then

$$\mathbb{E}[s^{X+Y}] = \mathbb{E}[s^X]\,\mathbb{E}[s^Y] \quad \Rightarrow \quad G_{X+Y} = G_X G_Y,$$

and likewise for M and φ. Convolution of distributions is multiplication of transforms. Taking logs, cumulants **add**: $K_{X+Y} = K_X + K_Y$.

(b) Moments come from derivatives.
- PGF: $G'(1) = \mathbb{E}[X]$, $G''(1) = \mathbb{E}[X(X-1)]$ (a *factorial* moment), so

$$\operatorname{Var}(X) = G''(1) + G'(1) - G'(1)^2.$$

- MGF: $M^{(k)}(0) = \mathbb{E}[X^k]$.
- CGF: $K'(0) = \mu$, $K''(0) = \sigma^2$, $K'''(0) = \mathbb{E}[(X-\mu)^3]$. Skewness is κ_3/σ^3 and excess kurtosis is κ_4/σ^4.

Because cumulants add over independent sums, the skewness of a sum of n i.i.d. variables is $n\kappa_3/(n\sigma^2)^{3/2} = \text{skew}/\sqrt{n}$ and excess kurtosis is kurt/n. That is the CLT in one line: higher cumulants die faster than the variance grows. (This is also why daily returns look fat-tailed but monthly returns look much more normal.)

(c) Uniqueness.
- A PGF is a power series, so its coefficients *are* the probabilities: equal PGFs means equal distributions.
- If M_X is finite on an open interval around 0, it determines the distribution (proof idea: M is then analytic in a strip, and $\varphi(t) = M(it)$ is its restriction to the imaginary axis).
- The CF always determines the distribution, via the **inversion formula**

$$P(a < X < b) + \tfrac{1}{2}[P(X=a) + P(X=b)] = \lim_{T\to\infty} \frac{1}{2\pi} \int_{-T}^{T} \frac{e^{-ita} - e^{-itb}}{it}\, \varphi_X(t)\, dt.$$

(d) Lévy's continuity theorem. $X_n \xrightarrow{d} X$ if and only if $\varphi_{X_n}(t) \to \varphi_X(t)$ for every t. (More precisely: if $\varphi_{X_n} \to \varphi$ pointwise and φ is continuous at 0, then φ is a CF and X_n converges in distribution to it. Continuity at 0 is exactly what rules out mass escaping to infinity.)

CLT proof sketch. Let X_i be i.i.d. with mean 0, variance 1. Taylor expand: $\varphi_X(t) = 1 - t^2/2 + o(t^2)$. Then

$$\varphi_{S_n/\sqrt{n}}(t) = \varphi_X\!\left(\tfrac{t}{\sqrt{n}}\right)^n = \left(1 - \frac{t^2}{2n} + o\!\left(\tfrac{1}{n}\right)\right)^n \to e^{-t^2/2},$$

which is the CF of $N(0,1)$. Lévy finishes the job. Notice that only two moments were used.

With infinite variance the expansion breaks: the Cauchy CF is $e^{-|t|}$, so the mean of n Cauchys has CF $(e^{-|t|/n})^n = e^{-|t|}$: the average of Cauchys is the same Cauchy. No LLN, no CLT.

1.3 Random sums and branching: composition

If N is a non-negative integer independent of the i.i.d. X_i, and $S = X_1 + \cdots + X_N$, condition on N (the tower property from yesterday):

$$G_S(s) = \mathbb{E}\big[\mathbb{E}[s^S \mid N]\big] = \mathbb{E}\big[G_X(s)^N\big] = G_N\big(G_X(s)\big).$$

Random sums are **composition**. Differentiating at $s = 1$ recovers Wald's $\mathbb{E}[S] = \mathbb{E}[N]\mathbb{E}[X]$ and the total-variance formula from yesterday's digest.

Two quick consequences:
- **Poisson thinning.** $N \sim \text{Poisson}(\lambda)$, each item kept w.p. p ($X_i \sim \text{Bernoulli}(p)$). Then $G_S(s) = e^{\lambda(1-p+ps-1)} = e^{\lambda p(s-1)}$, so the kept count is $\text{Poisson}(\lambda p)$. (Order flow arrivals split by venue or by side stay Poisson.)
- **Branching processes (Galton-Watson).** Each individual has offspring with PGF G. Generation n has PGF $G^{\circ n}$ (G composed n times). The extinction probability q is the **smallest root of** $q = G(q)$ **in** $[0, 1]$. Proof sketch: $q_n = P(Z_n = 0) = G(q_{n-1})$ with $q_0 = 0$; G is increasing and convex on $[0, 1]$ with $G(1) = 1$, so q_n increases to the first fixed point. If the mean offspring $G'(1) \le 1$ (and the process is not trivial) that fixed point is 1; if $G'(1) > 1$ there is a root below 1.
Example: offspring $0, 1, 2$ with probabilities $\frac{1}{4}, \frac{1}{4}, \frac{1}{2}$. Solve $q = \frac{1}{4} + \frac{1}{4}q + \frac{1}{2}q^2 \iff 2q^2 - 3q + 1 = 0 \iff q \in \{\frac{1}{2}, 1\}$. Extinction probability $\frac{1}{2}$.

1.4 Worked example: waiting time for HH

Toss a fair coin until two heads in a row. Let T be the number of tosses and $G(s) = \mathbb{E}[s^T]$. First-step analysis, where each toss contributes a factor s:
- First toss T (prob $\frac{1}{2}$): we are back at the start, contributing $\frac{s}{2}G(s)$.
- First toss H then H (prob $\frac{1}{4}$): done in 2, contributing $\frac{s^2}{4}$.
- First toss H then T (prob $\frac{1}{4}$): back at the start after 2, contributing $\frac{s^2}{4}G(s)$.

So $G = \frac{s}{2}G + \frac{s^2}{4} + \frac{s^2}{4}G$, giving

$$G(s) = \frac{s^2/4}{1 - s/2 - s^2/4}.$$

Expanding, $G(s) = \frac{1}{4}s^2 + \frac{1}{8}s^3 + \frac{1}{8}s^4 + \frac{3}{32}s^5 + \frac{5}{64}s^6 + \ldots$, i.e. $P(T = n) = F_{n-1}/2^n$ with F the Fibonacci numbers (the denominator is the Fibonacci recursion). Differentiating: $\mathbb{E}[T] = 6$ and $\text{Var}(T) = 22$. One function gave the whole distribution, the mean and the variance.

1.5 Pitfalls

1. $G''(1)$ **is not** $\mathbb{E}[X^2]$**.** It is $\mathbb{E}[X(X-1)]$. Forgetting the $+G'(1)$ in the variance is the most common slip.
2. **The MGF may not exist.** Cauchy: no MGF at all. Lognormal: all moments are finite, but $M(t) = \infty$ for every $t > 0$, and in fact the lognormal moments do **not** determine the distribution (other distributions share all of them). "Same moments" does not mean "same law" unless the MGF exists near 0. Use the CF when in doubt.
3. **Products need independence, and the converse fails.** $\varphi_{X+Y} = \varphi_X \varphi_Y$ can hold for dependent X, Y (there are explicit counterexamples), so a factorizing CF is not a test for independence. The joint CF $\mathbb{E}[e^{i(sX+tY)}] = \varphi_X(s)\varphi_Y(t)$ for all s, t is.
4. **Moment convergence vs distribution convergence.** $X_n \xrightarrow{d} X$ does not imply $\mathbb{E}[X_n] \to \mathbb{E}[X]$ (mass can escape: $X_n = n$ w.p. $1/n$, else 0). You need uniform integrability.
5. **Light tails from the MGF.** Finite MGF near 0 means exponentially decaying tails (by Chernoff: $P(X \geq a) \leq e^{-ta}M(t)$). Power-law tails (like many return series) have no MGF, so any argument that uses one silently assumes the tails away.

1.6 Interview question

> You have two six-sided dice. You may load each die however you like (each face gets any probability, the two dice need not be loaded the same way, and they are rolled independently). Can you make the sum uniformly distributed on $\{2, 3, \ldots, 12\}$?
>
> Follow-up: you may instead relabel the faces of two fair dice with positive integers (not necessarily distinct, not necessarily 1 to 6). Find a pair, other than the standard dice, whose sum has exactly the same distribution as two standard dice.

1.7 Solution

Set-up. Let die 1 show face i with probability p_i and die 2 with probability q_i, $i = 1, \ldots, 6$. Uniform on 11 values means $P(\text{sum} = k) = \frac{1}{11}$ for $k = 2, \ldots, 12$.

Approach 1: generating functions (the "real" proof). Define

$$A(s) = \sum_{i=1}^{6} p_i s^i = s\,a(s), \qquad B(s) = \sum_{i=1}^{6} q_i s^i = s\,b(s),$$

where a, b are polynomials of degree at most 5 with real coefficients. Independence gives $G_{\text{sum}} = AB$. We need

$$s^2 a(s) b(s) = \frac{1}{11}(s^2 + s^3 + \cdots + s^{12}) \quad \Longleftrightarrow \quad a(s)b(s) = \frac{1}{11} \cdot \frac{s^{11} - 1}{s - 1}.$$

Step 1: degrees. The right side has degree 10, so $\deg a + \deg b = 10$, forcing both to have degree exactly 5 (so $p_6, q_6 > 0$).

Step 2: roots. The roots of $(s^{11}-1)/(s-1)$ are the eleventh roots of unity other than 1: $e^{2\pi i k/11}$ for $k = 1, \dots, 10$. None of them is real (the only real eleventh root of unity is 1, which we removed; -1 is not one because 11 is odd).

Step 3: contradiction. a is a real polynomial of **odd** degree 5, so by the intermediate value theorem it has at least one real root. That real root would also be a root of ab, but ab has no real roots. Contradiction. **It is impossible.**

Approach 2: the 30-second interview trick. Only three sums are needed.
- $P(2) = p_1 q_1 = \frac{1}{11}$ and $P(12) = p_6 q_6 = \frac{1}{11}$.
- $P(7) \geq p_1 q_6 + p_6 q_1$ (ignoring the other ways to make 7).
- By AM-GM, $p_1 q_6 + p_6 q_1 \geq 2\sqrt{p_1 q_6 p_6 q_1} = 2\sqrt{(p_1 q_1)(p_6 q_6)} = 2\sqrt{\frac{1}{121}} = \frac{2}{11}$.

So $P(7) \geq \frac{2}{11} > \frac{1}{11}$. Impossible. The intuition: to make the extremes as likely as each other, both dice need weight at both ends, and the cross terms then over-produce the middle. The same argument kills two n-sided dice for every $n \geq 2$ (use the sum $n+1$).

Sanity check. The "closest" you can get is not close: with fair dice $P(7)/P(2) = 6$, and the bound says that ratio is always at least 2 under the constraint $P(2) = P(12)$. A quick numerical optimiser (minimise $\sum_k (P(k) - 1/11)^2$ over the two simplices) bottoms out at about 0.0028, never zero, consistent with the proof.

Follow-up: Sicherman dice. Now we keep each face probability $\frac{1}{6}$ and change the labels, so we want two polynomials with non-negative integer coefficients, each summing to 6 at $s = 1$, whose product equals

$$(s + s^2 + \cdots + s^6)^2.$$

Factor over the integers: $s + s^2 + \cdots + s^6 = s(1+s)(1+s+s^2)(1-s+s^2)$. So the product is

$$s^2 (1+s)^2 (1+s+s^2)^2 (1-s+s^2)^2,$$

and we must share these factors between two dice. Constraints:
- each die needs one factor s (no face labelled 0, since labels are positive),
- each die must evaluate to 6 at $s = 1$; at $s = 1$ the factors evaluate to $(1+s) \to 2$, $(1+s+s^2) \to 3$, $(1-s+s^2) \to 1$, so each die needs exactly one $(1+s)$ and one $(1+s+s^2)$.

The only freedom is the two copies of $(1-s+s^2)$. Standard dice give one to each die. Giving both to one die and none to the other:
- Die 1: $s(1+s)(1+s+s^2) = s + 2s^2 + 2s^3 + s^4$, faces $\{1, 2, 2, 3, 3, 4\}$.
- Die 2: $s(1+s)(1+s+s^2)(1-s+s^2)^2 = s + s^3 + s^4 + s^5 + s^6 + s^8$, faces $\{1, 3, 4, 5, 6, 8\}$.

(Expanding the product confirms it equals $(s + \cdots + s^6)^2$, so the sum distribution is identical.) And this is the **only** non-standard pair, since there was only one other way to place the factors. This is the kind of answer that shows the interviewer you see generating functions as an algebraic tool, not just a moment machine.

---

2. Derivatives: Repo, Specialness and the Cash-Futures Basis Trade

Yesterday we priced Treasury futures off the cheapest-to-deliver (CTD) bond and defined the implied repo rate (IRR). Today is the other half: the repo market itself, and the trade that connects the two, which is one of the largest leveraged trades in the financial system.

2.1 What a repo is

A **repurchase agreement** is a sale of a security today with an agreement to buy it back later at a fixed higher price. Economically it is a **collateralised loan**:
- The **repo** side (cash borrower) delivers bonds and receives cash.
- The **reverse repo** side (cash lender) receives bonds and lends cash.

If you borrow cash C for d days at repo rate r (act/360 in USD), the repurchase price is

$$C\left(1 + r\,\frac{d}{360}\right).$$

The cash lent is the bond's dirty value less a **haircut** (for example 2%), which protects the lender if the borrower defaults and the bond falls in value. Coupons paid during the repo are passed back to the bond's original owner ("manufactured" coupons), so the economic owner keeps the price risk and the coupon.

Who is in it and why.
- *Cash lenders:* money market funds, corporate treasuries, bank treasuries, the Fed's overnight reverse repo facility (as a floor). They want a safe, short, collateralised return.
- *Cash borrowers:* dealers financing inventory, and hedge funds financing leveraged long bond positions.
- *Securities borrowers:* anyone who is short a bond (a dealer hedging, a relative-value fund) must deliver it, so they reverse it in: lend cash, receive the bond, then sell it.

Venues. Tri-party repo (a clearing bank sits in the middle, general collateral), bilateral repo (two parties, specific bonds), and centrally cleared repo through FICC (including "sponsored" repo for hedge funds and money funds). SOFR, the benchmark USD rate, is a volume-weighted median of overnight Treasury repo transactions. The SEC's Treasury clearing mandate will push much more of this into central clearing (the current compliance dates are end-2026 for eligible cash trades and mid-2027 for repo).

2.2 General collateral vs special

- **General collateral (GC):** the lender accepts any Treasury from a broad basket. The rate is the market price of secured cash, close to SOFR.
- **Special:** the lender wants a *specific* bond (CUSIP), usually because they are short it and must deliver it. If many people want to borrow the same bond, its holders can demand to borrow cash very cheaply against it. The bond's repo rate trades below GC; the gap is **specialness**:

$$\text{specialness} = r_{\text{GC}} - r_{\text{special}}.$$

Specialness is income for the bond's owner: you can borrow cash at, say, 3.30% when GC is 3.80% and invest it at GC. That is why **on-the-run** bonds (the most recently auctioned, most liquid, most shorted) trade at a premium in price: the premium is roughly the present value of the expected future specialness.

How special can a bond get? The Treasury market fails charge (an annual penalty of $\max(3\% - \text{fed funds target lower bound}, 0)$ on a failed delivery) caps it: once the special rate falls far enough, it becomes cheaper for the short to fail than to borrow. With policy rates above 3% the fails charge is zero, so special rates can approach (but rarely go much below) zero.

2.3 The basis trade

The idea. From yesterday, holding the CTD to delivery and delivering it into a short futures position earns the implied repo rate. If you can finance the bond in repo at a rate **below the IRR**, you lock in the difference:

1. Buy the CTD (face N), financed in repo.
2. Sell $N \times \text{CF}/100{,}000$ futures (the conversion-factor hedge ratio, so a 1-tick move in the bond's futures-equivalent price is hedged).
3. At delivery, deliver the bond against the futures and repay the repo.

In price terms per 100 face: gross basis $= P - \text{CF} \cdot F$, carry = coupon accrual − financing, net basis = gross basis − carry. Holding to delivery, the gross basis converges to zero for the CTD, so

$$\text{P\&L per 100 face} = -\text{net basis} \quad (\text{computed with your own repo rate}).$$

Being long the bond and short the futures is called **"long the basis"**. It is also long the delivery options (the short futures holder chooses what and when to deliver), so a net basis close to zero means you are being given those options nearly for free.

Why does the opportunity exist? Asset managers and pension funds like to get duration through futures (cheap, little balance sheet). Someone has to be short those futures, and the natural seller hedges by owning cash bonds. That demand pushes futures rich relative to cash, so the IRR sits above the repo rate that hedge funds can finance at. The hedge

fund is effectively renting out its balance sheet and repo access. Estimates of the trade's size reached the high hundreds of billions to over a trillion dollars during 2024 to 2025, which is why regulators watch it closely.

2.4 Worked scenario: a basis trade into TYZ6

Illustrative numbers. Trade Thursday 15 Oct 2026, settlement Friday 16 Oct. Plan to deliver on Thursday 31 Dec 2026: $d = 76$ days.

The bond. A 7-year note, $4\frac{1}{8}\%$ coupon maturing 31 Aug 2033 (coupons 28 Feb and 31 Aug, so no coupon falls inside the trade). Price **100-20** $= 100.625$. Assume it is the CTD.

Conversion factor. Remaining maturity from 1 Dec 2026, rounded down to a quarter, is 6 years 9 months. The CME formula at 6% gives $\mathrm{CF} = 0.8971$.

Accrued interest (coupon period 31 Aug 2026 to 28 Feb 2027 is 181 days; semiannual coupon 2.0625):
- At settlement (46 days in): $\mathrm{AI}_0 = 2.0625 \times 46/181 = 0.52417$.
- At delivery (122 days in): $\mathrm{AI}_T = 2.0625 \times 122/181 = 1.39019$.

Dirty price today: $D_0 = 100.625 + 0.52417 = 101.14917$.

Futures. TYZ6 at **112-04+** $= 112.140625$. Then $\mathrm{CF} \cdot F = 0.8971 \times 112.140625 = 100.60135$.

Implied repo rate.

$$\text{Invoice} = \mathrm{CF} \cdot F + \mathrm{AI}_T = 100.60135 + 1.39019 = 101.99155,$$

$$\mathrm{IRR} = \left(\frac{101.99155}{101.14917} - 1 \right) \frac{360}{76} = 3.945\%.$$

Your financing. GC is 3.75%; as a hedge fund you pay 3.80% in sponsored repo.

Basis decomposition (per 100 face).

Item	Calculation	Value	In 32nds
Gross basis	$100.625 - 100.60135$	$+0.02365$	$+0.76$
Coupon accrual	$1.39019 - 0.52417$	0.86602	
Financing at 3.80%	$101.14917 \times 0.038 \times 76/360$	0.81144	
Carry	$0.86602 - 0.81144$	$+0.05458$	$+1.75$
Net basis	$0.02365 - 0.05458$	-0.03094	-0.99

The net basis is negative against *your* repo rate: the futures are rich by about one 32nd relative to your funding. That matches IRR (3.945%) above your repo (3.80%).

Sizing and P&L on $100M face.
- Futures: $100{,}000{,}000/100{,}000 \times 0.8971 = 897.1 \rightarrow$ **sell 897 TYZ6**.
- Expected P&L to delivery: $0.03094\% \times \$100\text{M} \approx$ **$30,900**. It does not depend on where rates go, provided you can hold to delivery.

(Check via cash flows: deliver at invoice 101.99155 per 100 and repay a repo loan of 98% of $\$101.149$M at 3.80% for 76 days. That gives about $\$47{,}200$ on your $2.02M haircut equity, which is the $30,900 plus the 3.80% you implicitly "earned" on your own equity. The net-basis framework charges financing on the full dirty value, i.e. it charges you the opportunity cost of your capital.)

Why it is levered. The edge is $\text{IRR} - \text{repo} \approx 14.5$ bp annualised. Capital required:
- Repo haircut 2% of $101.15M: $2.02M.
- Futures initial margin, assume $2,000 per contract: $1.79M.
- Total about $3.82M, i.e. leverage of about $26\times$ on a $101M position.

Return: $30{,}900/3.82\text{M} = 0.81\%$ over 76 days, about 3.9% annualised. At a 0% haircut (common in bilateral and sponsored repo for good clients) the capital is just the margin, about 1.7% over the period or 8.3% annualised at $56\times$ leverage. Funds push the leverage because the raw spread is tiny.

Specialness kicker. If the CTD goes special and your repo drops to 3.30% for the term, financing falls by $101.14917 \times 0.005 \times 76/360 = 0.10677$ per 100, an extra **$106,800**: more than triple the base P&L. Long-basis holders love a special CTD; anyone short the CTD (short basis) gets squeezed.

Stress: what March 2020 looked like. Suppose foreign reserve managers and others dump cash Treasuries, so cash cheapens versus futures and the gross basis widens by 16/32 (0.5 points):
- Mark-to-market loss: $0.5\% \times \$100\text{M} = \$500{,}000$, about 13% of your capital, even though the trade will still converge at delivery.
- CME raises initial margin 50%: another $897,000 of cash needed immediately.
- Your repo lender raises the haircut from 2% to 4%: another $2.0M.

You now need roughly $3.4M of fresh cash on $3.8M of capital. If you cannot find it, you must unwind: sell cash bonds and buy back futures, which widens the basis further for everyone else doing the same trade. That feedback loop is why the Fed bought Treasuries on a huge scale in March 2020.

2.5 What can go wrong

1. **Mark-to-market before convergence.** Convergence is only guaranteed at delivery. Before that, the basis can move against you and margin is paid daily.

2. **Funding (rollover) risk.** Many trades are financed in overnight repo against a 2 to 3 month futures horizon. If repo rates spike (September 2019, when overnight repo printed far above the Fed's target) or lenders pull back, carry turns negative.
3. **Margin and haircut procyclicality.** Exactly when volatility rises, CCPs and lenders demand more collateral.
4. **CTD switch.** If yields move far enough, another bond becomes CTD; your bond's basis then no longer converges to zero, and you own the switch option you may have under- or over-paid for.
5. **Delivery mechanics and squeezes.** If the CTD gets scarce (squeezed), shorts struggle to source it; the long basis wins, the short basis can lose badly.
6. **Crowding.** When everyone has the same trade, everyone unwinds at once (March 2020; basis and swap-spread stress also hit in April 2025 during the tariff-driven Treasury selloff).

2.6 Likely interview questions

1. **"Why does a bond trading special have a higher price than an otherwise identical bond?"** Owning it lets you borrow cash below GC. That cheap financing has value, and the price premium is roughly the present value of expected future specialness. As the bond rolls off the run, specialness fades and the premium decays.
2. **"You are long the basis. Are you long or short the delivery options, and what P&L do you expect if you hold to delivery?"** Long: you are short the futures, and the short chooses which bond and when to deliver. Holding to delivery, P&L is minus the net basis computed at your own repo rate; the delivery options are the upside on top of that if yields move enough to switch the CTD.
3. **"Futures look rich: IRR is 3.95% and repo is 3.80%. Why isn't this free money?"** It is close to riskless only if held to delivery *and* you can keep funding it. The risks are mark-to-market basis moves, margin and haircut hikes, overnight repo spikes on a term trade, and a CTD switch. The edge (here 14 bp) is paying you for balance sheet, leverage and liquidity risk.

---

3. Algorithms & Data Structures: Binary Search on the Answer

3.1 Problem: Split Array Largest Sum

Given an array `nums` of n non-negative integers and an integer k, split the array into exactly k non-empty **contiguous** pieces so that the largest piece sum is as small as possible. Return that minimum largest sum.

Constraints: $1 \le n \le 10^5$, $0 \le$ `nums[i]` $\le 10^9$, $1 \le k \le n$. (The total can reach 10^{14}, so use 64-bit integers.)

Examples

- `nums = [7,2,5,10,8], k = 2` → `18` (split `[7,2,5] | [10,8]`, sums 14 and 18).
- `nums = [1,2,3,4,5], k = 2` → `9` (`[1,2,3] | [4,5]`).
- `nums = [1,4,4], k = 3` → `4`.

Quant framing: assign a sequence of orders (or a day's time buckets) to k traders or machines, keeping each one's contiguous block, so that the busiest one has as little load as possible.

3.2 Brute force and the DP

- **Brute force:** choose $k-1$ cut points among $n-1$ gaps: $\binom{n-1}{k-1}$ splits, exponential.
- **DP:** let $dp[j][i]$ be the best largest-sum for the first i elements in j pieces:

$$dp[j][i] = \min_{p<i} \max\left(dp[j-1][p],\ \text{sum}(p..i-1)\right).$$

With prefix sums that is $O(kn^2)$ time, fine for $n = 1000$ but hopeless for $n = 10^5$. (The C++ file keeps this DP as a test oracle.)

3.3 The insight: flip optimisation into a yes/no question

Instead of asking "what is the best largest sum?", ask: **"Given a cap C, can we split into at most k pieces with every piece sum $\le C$?"**

Two facts make this powerful:

1. **The yes/no question is easy (greedy, $O(n)$).** Scan left to right, extending the current piece while it stays $\le C$; when the next element would overflow, start a new piece. Greedy is optimal because closing a piece later never hurts: any valid split can be transformed into the greedy one by pushing each cut as far right as allowed, without increasing the number of pieces (an exchange argument).
2. **The answer is monotone in C.** If cap C is feasible then any $C' > C$ is feasible. So the feasibility predicate looks like `F F F ... F T T T ... T`, and we want the first `T`.

"At most k" is enough: with non-negative values and $k \le n$, a split into fewer pieces can be refined into exactly k by splitting off single elements, which never increases any piece sum.

Search range. The answer is at least $\max(\texttt{nums})$ (some piece contains the biggest element) and at most $\sum \texttt{nums}$ (one piece). Binary search the first feasible value in $[\max, \sum]$.

Trace for `[7,2,5,10,8], k = 2`: range $[10, 32]$.

lo	hi	mid	greedy pieces	feasible?
10	32	21	`[7,2,5] [10,8]`	yes, hi = 21
10	21	15	`[7,2,5] [10] [8]`	no (3 pieces), lo = 16
16	21	18	`[7,2,5] [10,8]`	yes, hi = 18
16	18	17	`[7,2,5] [10] [8]`	no, lo = 18

`lo == hi == 18`. Done.

Complexity: $O(n \log S)$ time where $S = \sum \texttt{nums}$ (at most about 47 iterations for $S \le 10^{14}$), $O(1)$ extra space.

The general pattern, "binary search on the answer", applies whenever the objective is a min-max or max-min and feasibility is monotone: ship packages within D days, Koko eating bananas, minimum time for m machines to finish t jobs, the largest minimum distance when placing m items, or (with doubles) finding an implied volatility, since option price is monotone in vol.

3.4 C++ solution

```cpp
#include <algorithm>
#include <numeric>
#include <vector>
using namespace std;

long long splitArray(const vector<long long>& nums, int k) {
    // Can we split into at most k pieces, each with sum <= cap?
    auto feasible = [&nums, k](long long cap) {
        int pieces = 1;
        long long running = 0;
        for (long long x : nums) {
            if (running + x > cap) {   // x does not fit: close the piece
                ++pieces;
                running = x;           // x <= cap is guaranteed by lo below
                if (pieces > k) return false;
            } else {
                running += x;
            }
        }
        return true;
    };

    long long lo = *max_element(nums.begin(), nums.end());
    long long hi = accumulate(nums.begin(), nums.end(), 0LL);

    while (lo < hi) {                      // find first cap with feasible == true
        long long mid = lo + (hi - lo) / 2;
        if (feasible(mid)) hi = mid;       // mid works: answer <= mid
        else               lo = mid + 1;   // mid fails: answer > mid
    }
    return lo;
}
```

The full file `2026-10-15.cpp` adds a `main` with the examples, a 64-bit overflow test (three values of 10^9), and a randomized cross-check of 2000 small cases against the $O(kn^2)$ DP. Everything passes.

3.5 Walkthrough of the C++

- `const vector<long long>& nums`: pass by const reference. No copy of a potentially 10^5-element vector, and the compiler stops you from modifying it. Use `long long` (at least 64 bits) because sums reach 10^{14}, far beyond `int`'s roughly 2.1×10^9.
- **The lambda** `[&nums, k](long long cap) { ... }`: the capture list says what the lambda can see from the enclosing scope. `&nums` captures by reference (no copy), `k` by value (an `int` is cheap). Its type is unnamed, so we store it with `auto`. It is called like a function: `feasible(mid)`. Keeping the predicate as a lambda right next to the search keeps the "monotone predicate + binary search" structure obvious.
- **Range-based for,** `for (long long x : nums)`: iterates over elements by value; for `long long` that is cheaper than a reference. For heavier element types, write `const auto& x`.
- `*max_element(nums.begin(), nums.end())`: `max_element` (from `<algorithm>`) returns an **iterator** to the largest element, not the value, so we dereference with `*`. On an empty range it returns `end()`, and dereferencing that is undefined behaviour; the constraints guarantee $n \geq 1$.
- `accumulate(nums.begin(), nums.end(), 0LL)`: from `<numeric>`. The type of the initial value decides the accumulator type. Writing `0` instead of `0LL` would sum in `int` and overflow silently. This is a classic bug.
- `lo + (hi - lo) / 2`: avoids overflow of `lo + hi` (irrelevant at 10^{14} in 64 bits, but a good habit, and essential with `int`). Integer division rounds down, so `mid < hi` always, and both branches strictly shrink the interval: `hi = mid` or `lo = mid + 1`. Pairing "round down" with `hi = mid` is what guarantees termination; if you ever write `lo = mid`, you must round up instead.
- **Loop invariant**: the answer is always in `[lo, hi]`. It starts true (the answer lies between max and sum) and each branch preserves it, so when `lo == hi` that value is the answer. Stating the invariant out loud is the fastest way to convince an interviewer your binary search has no off-by-one.
- **Testing (`main`)**: an aggregate `struct Test` initialised with brace lists, `assert` from `<cassert>` (active unless `NDEBUG` is defined), and `std::mt19937` with `uniform_int_distribution` for reproducible random tests. Cross-checking a fast solution against a slow, obviously correct one is the single best habit for interview prep and for production quant code alike.

3.6 Variations to be ready for

- **Real-valued version** (e.g. minimise the maximum average, or solve for implied vol): loop a fixed number of times (say 100) or until `hi - lo < eps`, with `lo = mid` / `hi = mid` and no `+1`.
- **Return the actual split**: run `feasible(answer)` once more and record the cut positions.

- **Negative numbers allowed**: monotonicity of the greedy breaks (a piece's sum is no longer monotone as it grows), so binary search on the answer fails; fall back to DP.
- **Faster DP**: the cost is monotone in the cut position, so Knuth-style or divide-and-conquer optimisation brings the DP down to $O(kn \log n)$ or better.